

Amanda Andrade Abreu
Guilherme Amaral Giffoni

Material de Apoio – Seminário de Prolog

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Instituto de Ciências Exatas e Informática

Professor: Dr. Marco Rodrigo Costa

Belo Horizonte
2026

Sumário

1	INTRODUÇÃO	3
1.1	Philippe Roussel	3
1.2	Alain Colmerauer	3
2	HISTÓRICO	5
2.1	Antecedentes pré-1972	5
2.2	Nascimento do Prolog	5
3	PARADIGMAS DA LINGUAGEM	7
4	CARACTERÍSTICAS MARCANTES	9
4.1	Comentários	9
4.2	Fatos	10
4.3	Perguntas	10
5	LINGUAGENS RELACIONADAS	13
6	EXEMPLO DE PROGRAMAS EM PROLOG	15
7	CONSIDERAÇÕES FINAIS	17
	REFERÊNCIAS	19

1 Introdução

Prolog foi inventada em Marseilles (na França), em 1972. Philippe Roussel escolheu o nome como uma abreviação para “**programação lógica**”¹, por sugestão de sua esposa no outono de 1972. Pode-se dizer que Prolog foi a prole de um casamento bem-sucedido entre processamento de linguagem natural e prova de teorema automatizada. A ideia de usar uma linguagem natural como o francês para raciocinar e comunicar diretamente com um computador parecia uma ideia *maluca*, apesar disso, foi a base para o projeto definido por Alain Colmerauer no verão da década de 1970. (COLMERAUER; ROUSSEL, 1993)

Notavelmente, Prolog foi peça central do IBM Watson, que em novembro de 2013, venceu os melhores da categoria no jogo televisivo *Jeopardy!*, sem conexão com a internet, e precede em 4 anos os modelos de IA baseados em *transformers* (aprendizado profundo). (RESEARCH, 2013)

1.1 Philippe Roussel

Philippe Roussel

1.2 Alain Colmerauer

Alain Colmerauer, foi um Doutor em ciência da computação, professor na Universidade de Aix-Marselha, e co-criador do Prolog, nascido em 24 de janeiro de 1941 em Carcassonne. Em 2000, a sua empresa de software se especializava em análise e gerência de riscos financeiros e hipoteca. E em 2005, foi comprada pela Experiean, que se tornou parte do negócio de suporte a decisões empresariais, Experian-Scorex. (FISHER, 2017)

Prêmios recebidos:

- Pomme d’Or du Logiciel Francais (1982)
- Michel Monpetit Award (1986)
- Chevalier de la Legion d’Honneur (1986)
- ACP Research Excellence Award (1991)

¹ em francês: “**programmation en logique**”.

2 Histórico

2.1 Antecedentes pré-1972

Prolog é descendente de três principais linhas de pesquisa, sendo elas: PAT (prova automática de teoremas), processamento de linguagem natural e IA. (KÖRNER et al., 2022)

O campo de pesquisa da inteligência artificial nasce por volta de 1956, e deu impulso para que fossem criadas linguagens de programação funcionais, como o LISP, criado em 1958 por McCarthy.

O PAT teve notória presença em um trabalho seminal publicado por Alan Robinson em 1965 onde descrevia lógica computacional baseada no princípio da resolução. O que é o coração dos procedimentos de inferência adotados à programação de lógica.

Enquanto isso, Alain Colmerauer tentava automatizar a conversação homem-máquina, o que o levou a desenvolver Q-systems que serviu para traduzir de inglês para francês os relatórios meteorológicos canadenses. Tudo isso poderia ser escrito em lógica, e inspirado nele, dentre outras coisas, os levou a criar Prolog.

2.2 Nascimento do Prolog

3 Paradigmas da Linguagem

4 Características Marcantes

Na experiência do autor do livro (citar livro), Prolog é uma linguagem que programadores iniciantes acham ser mais compreensiva que programas equivalentes em linguagens “convencionais”. Em contra-partida, programadores experientes nessas mesmas linguagens, apesar de estarem mais bem preparados para os conceitos abstratos que sondam as LPs, podem achar Prolog uma linguagem difícil de se acostumar, e precisam de muito convencimento até considerarem a linguagem como uma ferramenta de programação útil.

Desde seus começos ao redor de 1970, Prolog foi escolhido por muitos programadores para aplicações de computação simbólica, como:

- Lógica matemática
- Resolução de problemas abstratos
- Entender linguagem natural
- banco de dados relacional
- Implementar automação
- Resolução de equações simbólicas
- Análise de estrutura bioquímica
- E várias áreas de inteligência artificial

Prolog permite que um computador seja usado como um armazem de fatos e regras, nos permite a fazer inferências de um fato para outro, encontrar os valores da variáveis pode nos levar a uma dedução lógica.

4.1 Comentários

```
% esse é um comentário de uma linha só (lembra LaTeX)
mãe(roberta, joaquina) % roberta é mãe de joaquina

/*
rosas são vermelhas
violetas são azuis
comentário multilinha
adoro cuzcuz
*/
```

4.2 Fatos

Para expressar a relação: “João gosta de Maria”, escrevemos em Prolog:

```
gosta(joão, maria).
```

A ordem dos fatores é arbitrária, mas deve respeitar alguma ordem determinada pelo programador, para manter consistência. No exemplo, levemos em consideração que a pessoa que gosta, seja o primeiro “elemento” do que está entre parênteses. Que não é a mesma coisa que:

```
gosta(maria, joão).
```

Para demonstrar com mais clareza ainda como a consistência é importante na linguagem, observe a seguinte definição de um *fato*:

```
pai(tom, william).
```

Podemos assumir que *tom* é pai de *william*, mas também podemos interpretar que os dois possuem o mesmo pai. A linguagem não restringe a semântica, que por sua vez, é responsabilidade do programador.

Outros exemplos, com as possíveis interpretações:

```
valioso(diamante).      % Diamante é valioso.  
mulher(jane).            % Jane é uma mulher.  
possui(jane, diamante).  % Jane possui diamantes  
pai(alberto, maria).     % Alberto é pai de Maria  
da(alberto, livro, maria). % Alberto deu um livro para Maria
```

Note que cada fato descrito acima, termina com um ponto final.

Vamos agora dar as terminologias corretas para essa construção; os *objetos* entre parênteses (de vermelho) são os argumentos, e o nome da relação que vem antes (de azul), é chamado de *predicado*.

O coletivo de fatos é chamado de *base de dados*.

4.3 Perguntas

Tabela 1 – Análise SWOT

Pontos fortes: • Sintaxe e semântica simples • Estruturas de dados persistentes e imutáveis, com ponteiros “declarativos” • Precisão aritmética arbitrária • Segurança (garbage collection, exceções Null-Pointer inexistentes, ...) • Inferência eficiente, casamento de padrões e unificação • Indexação e tabelamento eficiente • Desenvolvimento e debugging rápido com REPL (loop de leitura, execução, impressão) • Aplicações de sucesso: – Análise de programas – Processamento de linguagem natural – Sistemas especialistas, PAT (Prova automática de teoremas)	Pontos fracos: • Sintaticamente diferente das LPs “tradicionais”, não é uma linguagem <i>mainstream</i> • Curva de aprendizado íngreme; iniciantes podem facilmente escrever programas que entram em loop ou consomem muitos recursos computacionais • Tipagem estática • Portabilidade limitada • Bibliotecas/pacotes: avaliabilidade e gerência • IDEs e ferramentas de desenvolvimento possuem capacidades limitadas em algumas áreas (como a refatoração) • Desenvolvimento de interfaces de usuário
Oportunidades: • Novas áreas de aplicação – IA neurosimbólica – IA explicável e verificável – <i>Big Data</i> • Novas funcionalidades e desenvolvimentos – Raciocínio probabilístico – Paralelismo (como é feito em Lean) – Compilador JIT – Integrando resolução de ASPs (Programação de conjunto de resposta) e SATs (Problema de satisfatibilidade booliana) ou SMTs (Teorias de satisfatibilidade módulo).	Ameaças • <i>User base</i> baixa, quando comparada com outras LPs • Comunidade fragmentada • Parte ativa da comunidade com novas implementações e funcionalidades constantemente desenvolvidas criam fragmentação • Novas linguagens de programação • A percepção de ser uma LP “antiga” • Imagem errada por ensino “raso” da linguagem • O mundo pós-desktop das aplicações web em JavaScript

Fonte: (KÖRNER et al., 2022), Adaptado

5 Linguagens Relacionadas

- PLANNER é uma linguagem influenciadora
- Bend (linguagem pt-br) é provavelmente uma linguagem de alguma forma relacionada, apesar de talvez não ter noção disso; em particular, Bend 2
- Linguagens OOP notórias como Java e C++ parecem ser opostas

6 Exemplo de Programas em Prolog

7 Considerações Finais

Referências

COLMERAUER, A.; ROUSSEL, P. The birth of prolog. *SIGPLAN Not.*, Association for Computing Machinery (ACM), v. 28, n. 3, p. 37–52, mar. 1993.

FISHER, L. M. *In Memoriam Alain Colmerauer: 1941-2017*. 2017. Website. Disponível em: <https://cacm.acm.org/news/in-memoriam-alain-colmerauer-1941-2017/>.

KÖRNER, P. et al. *Fifty Years of Prolog and Beyond*. 2022. Disponível em: <https://arxiv.org/abs/2201.10816>.

RESEARCH, I. *Watson and the Jeopardy! Challenge*. 2013. Website. Disponível em: <https://www.youtube.com/watch?v=P18EdAKuC1U>.